



Project Summary & Motivation

Summary

We created a pipeline for generating and cleaning training data. Then built a simple neural net for analyzing a chess board and predicting the next best move. From our basic architecture, we experimented with hyperparameter variations to test how certain changes impact performance. After we took our best model from testing and used transfer learning to continue training our best model on new datasets to improve its chess capabilities.

Motivation

The motivation for our project was to improve on an existing neural network using novel machine learning techniques. And understand how a models architecture impacts its performance in a complicated environment like chess. Moreover, we were interested in the idea of transfer learning, being able to take a pretrained model and adapt it to a similar yet specific application.

Project Overview

Data Sources

- Generated our own training data by having two Stockfish models play each other and record their games.
- Generated late-game data by removing the first 10 moves.
- Games from the July 2014 Lichess dataset, filtered Blitz and Classical time controlled with at least 2000 Elo.

Model Approach

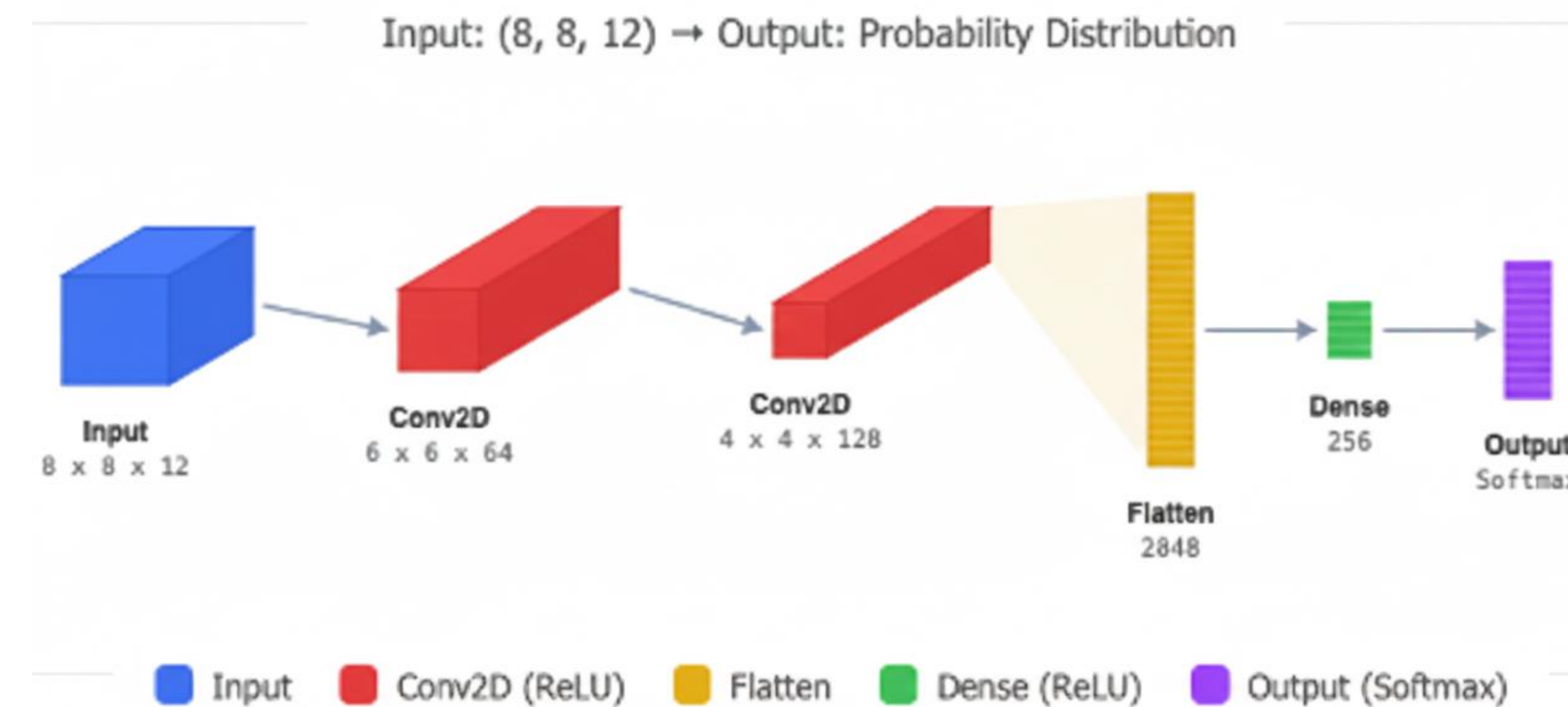
We used supervised learning for move prediction. For our transfer learning models, we used a similar supervised learning method with traditional transfer learning techniques for deeper learning on new datasets.

Results

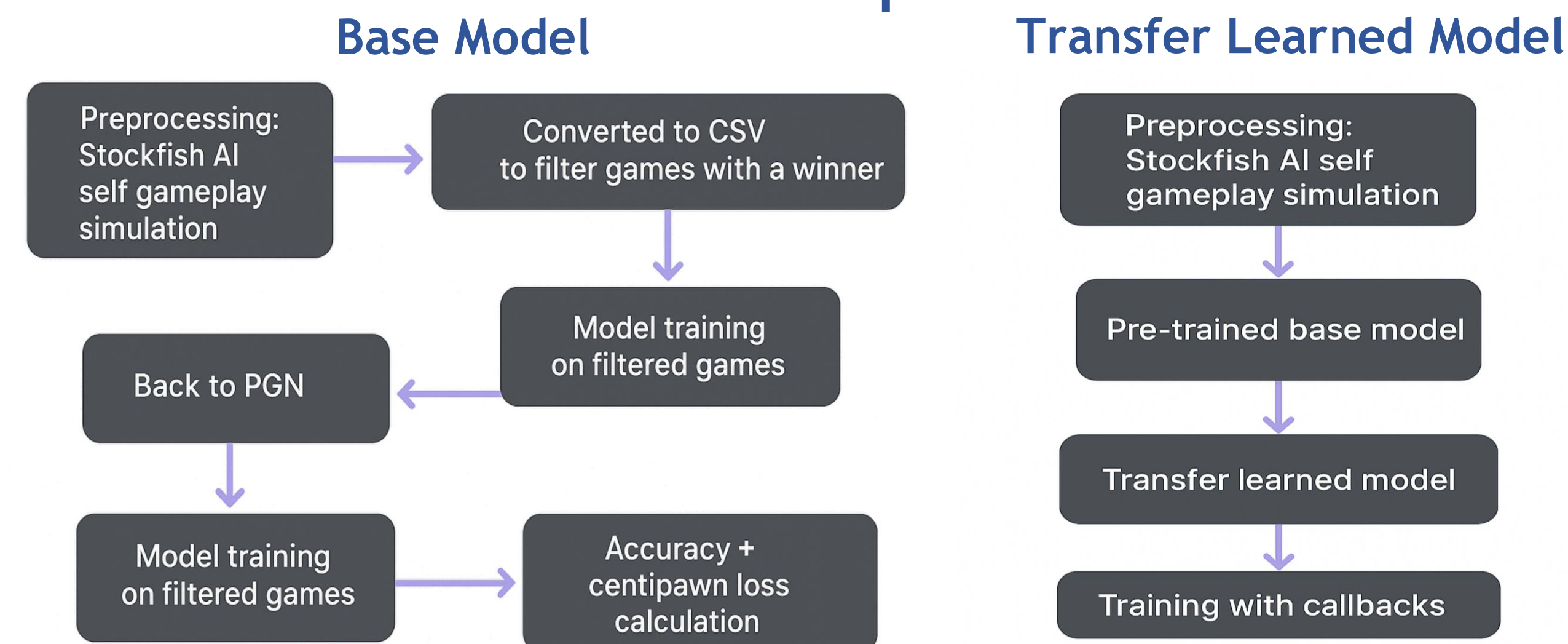
While experimenting with variations of our base model we found that using two convolutional layers with increasing filter sizes along with the ReLu activation function performed best.

Our chess model is able to play against Stockfish at 2000 Elo. Our original model was able to play with a Centipawn accuracy score of 79% with 76% loss. And our best transfer learning model was able to play with a Centipawn accuracy score of 77% with 70% loss.

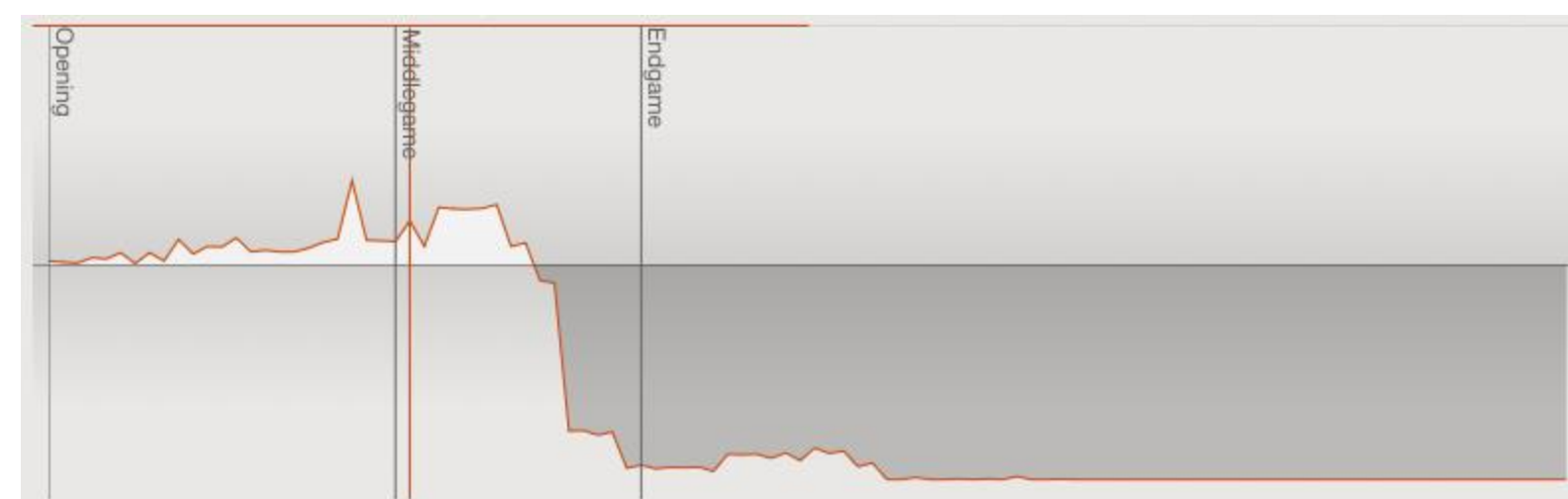
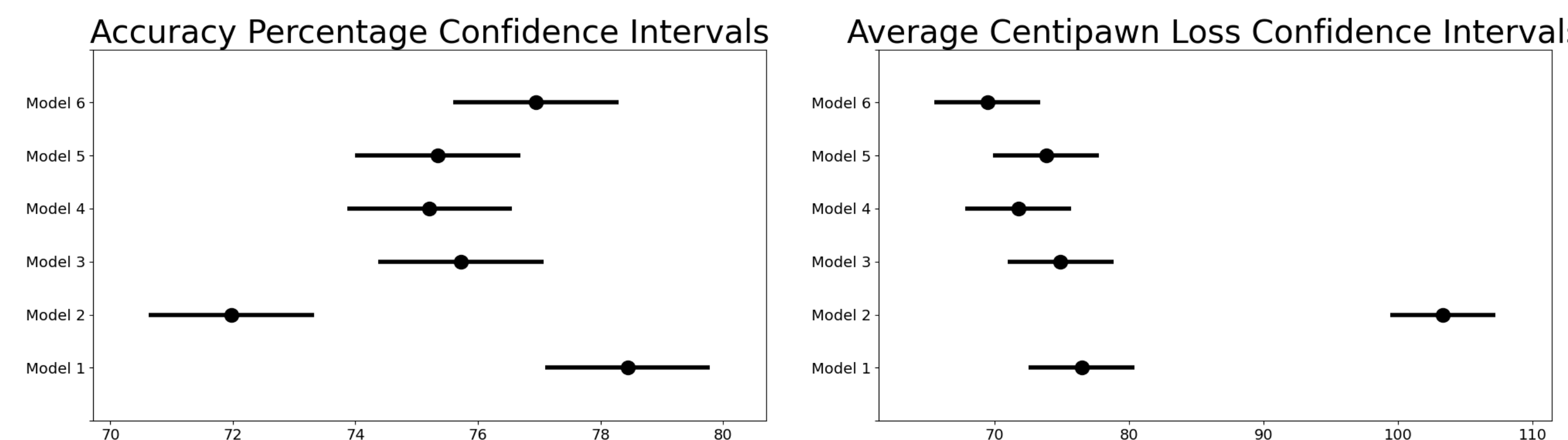
Model Architecture



Data Pipeline



Results



Methods & Technologies

Framework and Input Design

- Built our model using TensorFlow.
- Input layer is 8×8×12 to capture the total state of the chess board for both sides' six unique pieces.

Convolutional Layers

- Increasing size of convolution layers captures relationships between adjacent squares.
- Flatten layer converts the 4×4×128 vector into a 1×2048 vector for displaying each feature.

Dense Layers

- First dense layer performs holistic board evaluation that learns the spatial patterns extracted in prior layers.
- Second dense layer outputs a probability distribution of all possible legal moves the model has seen.

Transfer Learning Architecture Modifications

- Froze the convolutional and flatten layers.
- Added three dense layers with dropout layers in between them.
- Dense layers are larger to handle more complex spatial relationships
- Dropout layers prevent over-reliance on specific feature combinations, allowing the model to generalize better with greater spatial complexity.

Centipawn Scoring System

- Focused primarily on accuracy and loss scores generated by the system.
- Both measures rely on comparing player moves to best moves found by Stockfish.
- Centipawn average loss calculated from the difference between player moves and Stockfish recommendations.
- Centipawn accuracy score represents the percentage of moves that align with Stockfish's choices.

PGN (Portable Game Notation)

- Standard file format for recording chess games.
- Contains all moves present in a game along with information about the players.

References

- Lichess Dataset - <https://database.lichess.org/>
- Lichess Accuracy Calculations - [Accuracy](#)
- TensorFlow Transfer Learning Documentation - [TF Hub](#)
- Inspiration for Base Chess Engine - [Chess Engine](#)